gemstone-py User Manual

Core APIs, transaction model, persistence helpers, and web integration

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

User Manual

This manual explains the public surface of `gemstone-py` from the point of view of a user who wants to build reliable code, not just make a demo print a nice dictionary once.

The Mental Model

Three ideas matter more than the rest:

1. Python code drives the application.
2. GemStone stores the persistent state.
3. Transactions are explicit enough that you can explain them to another human.

`gemstone-py` is not trying to hide GemStone completely. It gives you a Pythonic surface, but it still wants you to understand:

- when sessions are opened and closed
- when transactions commit or abort
- which data structures are persisted in GemStone
- what happens when two sessions try to change the same thing

That restraint is a strength. The library does not pretend that distributed state is a teddy bear.

Core Building Blocks

`GemStoneConfig`

`GemStoneConfig` is the runtime configuration object. In most applications you will construct it from the environment:

```
from gemstone_py import GemStoneConfig

config = GemStoneConfig.from_env()
```

Use it when:

- you want a single source of connection settings
- you need to pass the same config into sessions, stores, or web providers
- you want to validate missing credentials early

`GemStoneSession`

This is the direct session object. Use it when you want explicit session and transaction control.

Typical pattern:

```
from gemstone_py import GemStoneSession, TransactionPolicy

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    print(session.eval("System myUserProfile userId"))
```

Useful methods include:

- `eval(...)`

  Evaluate Smalltalk code directly.

- `commit()`

  Commit the current transaction.

- `abort()`

  Abort the current transaction.

- `global_get(...)`

  Resolve named objects from the repository.

When to use it:

- scripts
- lower-level tooling
- integration code that must control failure behaviour precisely

`session_scope(...)`

This is the higher-level unit-of-work helper. It is usually the right answer for application code.

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    ...
```

Why it exists:

- it makes commit-on-success intent obvious
- it composes naturally with service-layer code
- it avoids sprinkling `commit()` everywhere like nervous confetti

`AsyncSession`

`gemstone_py.aio.AsyncSession` is the async facade for applications built on `asyncio`, FastAPI, or Starlette-style request handling.

It does not make GCI itself async. Instead, it runs all calls for one session on one dedicated worker thread. That keeps the GemStone session on its owning thread while letting your event loop continue serving other work.

```
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession

config = GemStoneConfig.from_env()

async with AsyncSession.connect(config=config) as session:
    value = await session.eval("3 + 4")

    async with session.transaction():
        root = session.root()
        await root.set("AsyncManualExample", {"value": value})
```

Use `AsyncSession` when:

- request handlers are already async
- you want FastAPI dependency injection
- you need to keep blocking GCI work out of the event loop

The runnable repository example is `examples/async_features/session_root_and_collection.py`.

Transaction Policies

`TransactionPolicy` exists so you do not have to guess what a context manager will do.

The main values are:

- `MANUAL`
- `COMMIT_ON_SUCCESS`
- `ABORT_ON_EXIT`

Recommended use:

Situation	Policy
low-level explicit control	`MANUAL`
script that writes data	`COMMIT_ON_SUCCESS`
read-only inspection	`ABORT_ON_EXIT`
request/response web unit of work	`COMMIT_ON_SUCCESS` via `session_scope(...)` or request integration

`PersistentRoot`

`PersistentRoot` is the friendliest entry point into repository-backed state.

The default instance is the user's `UserGlobals` dictionary:

```
from gemstone_py.persistent_root import PersistentRoot

root = PersistentRoot(session)
root["CustomerCount"] = 12
```

Other dictionaries are also available:

- `PersistentRoot.globals(session)`
- `PersistentRoot.published(session)`
- `PersistentRoot.session_methods(session)`

Use `PersistentRoot` when:

- you want a stable named entry point
- you are storing dictionaries, counters, queues, or domain objects
- you want something simple and direct

Use something more specialized when:

- you need indexed search across many rows -> `GSCollection`
- you want a file-like key/value store abstraction -> `GStore`
- you want append-only event style logging -> `ObjectLog`

Good Practices With `PersistentRoot`

- keep the top-level key space tidy
- use descriptive names
- group related data under one top-level bucket when it makes sense
- do not turn `UserGlobals` into a junk drawer with no naming discipline

GSCollection

`GSCollection` is the indexed collection helper. It is the right tool when you need more than "look up a dictionary by a single well-known key."

Use it for:

- indexed search
- filtering by stored attributes
- application collections with repeatable lookup patterns

Typical pattern:

```
from gemstone_py.gsquery import GSCollection

people = GSCollection("People", config=config)
people.insert({"@name": "Tariq", "@city": "London"})
people.add_index("@city")
matches = people.search("@city", "eql", "London")
```

Think of it as the package's middle ground between:

- raw repository objects
- and a full ORM that wants to restructure your life

Typed Queries

`GSCollection.query(...)` can accept a Python `Protocol` as a type witness. The query still runs against GemStone indexed paths, but your Python code gets attribute names and better editor help.

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class BlogPostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config).query(BlogPostRecord)
published = posts.where(lambda post: post.status == "published").all()

for post in published:
    print(post.title)
```

The lambda is a query builder expression, not a live object callback. `post.status` becomes the GemStone ivar path `@status`.

See `examples/typed_access/typed_oops_and_queries.py` for a runnable typed query and `examples/typed_access/simple_blog_queries.py` for importable helper functions.

Typed OOPs and Lifetime Handles

Raw integer OOPs remain supported for compatibility. New code can opt into typed or managed wrappers when the extra structure is useful.

`TypedOop[T]` carries a phantom Python type:

```
from typing import Protocol
from gemstone_py import GemStoneSession, gemstone_class

@gemstone_class("Date")
class GemStoneDate(Protocol):
    @property
    def printString(self) -> str: ...

with GemStoneSession(config=config) as session:
    today = session.execute_typed("Date today", GemStoneDate)
    print(today.proxy().printString)
```

For object lifetime, use `execute_managed(...)` when a raw OOP should stay in GemStone's export set while Python holds a handle:

```
with GemStoneSession(config=config) as session:
    collection = session.execute_managed("OrderedCollection new")
    collection.send("add:", session.new_string("kept alive"))
    print(collection.print_string())
    collection.close()
```

Use `session.handle(oop)` when you already have a raw OOP and want an explicit scope:

```
with session.handle(raw_oop) as handle:
    print(handle.send("printString"))
```

`execute()` and `perform()` still return raw OOP integers. The managed variants are additive and make lifetime intent visible.

The runnable repository example is `examples/lifetime/managed_oop_handles.py`.

GStore

`GStore` is a GemStone-backed key/value store with its own transactional behaviour around store operations.

Use it when:

- you want a store-like API
- you want separate named stores
- you want a compact way to persist structured payloads

Typical pattern:

```
from gemstone_py.gstore import GStore

store = GStore("inventory.db", config=config)
store["sku:123"] = {"name": "Hat", "stock": 8}
print(store["sku:123"])
```

`GStore` is especially handy in examples, utilities, and small application components that want a store-shaped abstraction without building a full domain model first.

`ObjectLog`

`ObjectLog` is the package's event-log helper. It is good for:

- audit trails
- structured append-only logging
- "what happened?" questions after a workflow ran

Typical pattern:

```
from gemstone_py.objectlog import ObjectLog

log = ObjectLog(config=config)
log.info("user_created", {"user": "tariq", "source": "manual"})
for entry in log.entries():
    print(entry)
```

This is not a replacement for Python logging. It is a repository-resident event log for when the record should live with the data.

Concurrency Helpers

The concurrency helpers give you shared, repository-backed coordination primitives:

- `RCCounter`
- `RCHash`
- `RCQueue`

These are useful when you need shared mutable state between sessions and want the concurrency semantics to live in GemStone instead of in a Python process.

Use them carefully:

- they are powerful
- they make contention visible
- they will teach you honesty about multi-session behaviour

The package also includes:

- nested transaction helpers
- conflict detection helpers
- instance listing utilities

Commit Conflicts

When two sessions modify the same object and both try to commit, GemStone raises a conflict. The second committer gets a `CommitConflictError`.

This is not a bug. It is the correct behaviour of an optimistic concurrency system. The right response is:

1. catch `CommitConflictError`
2. call `session.abort()` to reset the transaction
3. reload the data
4. reapply the change (with a narrowed scope if possible)
5. retry the commit

Keep retries bounded. Log them. If you see frequent conflicts on the same object, that is a signal the write pattern needs rethinking — not that the concurrency model is wrong.

```
from gemstone_py import GemStoneSession, TransactionPolicy
from gemstone_py.concurrency import CommitConflictError

for attempt in range(3):
    with GemStoneSession(config=config,
                         transaction_policy=TransactionPolicy.MANUAL) as session:
        try:
            counter = session.global_get("Visits")
            session.eval(f"Visits := {counter + 1}")
            session.commit()
            break
        except CommitConflictError:
            session.abort()
            if attempt == 2:
                raise
```

Web Integration

The web integration lives in `gemstone_py.web`.

The main pieces are:

- `install_flask_request_session(...)`
- `GemStoneSessionPool`
- `GemStoneThreadLocalSessionProvider`
- `session_scope(...)`

One Session Per Request

If you are building a Flask app, use the request integration so the request lifecycle decides the final transaction outcome.

That avoids the worst class of web persistence bugs:

- request handled an exception
- response still rendered
- persistence layer silently committed partial work anyway

The package already corrected this behaviour at the framework layer, so use it.

Pool vs Thread-Local

Use `GemStoneSessionPool` when:

- you have multiple request workers
- you want bounded reuse
- you want production-friendly pooling semantics

Use `GemStoneThreadLocalSessionProvider` when:

- your threading model is simple and stable
- one session per thread is the clearest fit

FastAPI

FastAPI integration lives in `gemstone_py.aio.fastapi`.

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession
from gemstone_py.aio.fastapi import session_dependency

app = FastAPI()
get_gemstone = session_dependency(config=GemStoneConfig.from_env())
```

```
@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone)):
    return {"result": await session.eval("3 + 4")}
```

Use this when your web stack is async-first. Use the Flask integration when your application is Flask-first.

The runnable repository example is `examples/fastapi/app.py`.

Benchmarks and Build Lanes

This package has both examples and maintained operational lanes. Keep them separate in your head.

Examples are for:

- learning
- translation exercises
- inspection

The benchmark lane is for:

- reproducible measurement
- stored JSON reports
- baseline comparison
- GitHub workflow enforcement

The CLI is:

```
gemstone-benchmarks --entries 500 --search-runs 20
```

Release and Packaging Surface

The public install surface is the `gemstone_py` package.

The package now has:

- PyPI publishing
- TestPyPI rehearsal
- post-release verification against real PyPI
- installed-artifact API contract checks
- local PyPI/TestPyPI index verification with `gemstone-publish-verify`
- optional PyO3 native wheels through `gemstone-py[fast]`

That means you can treat the package as a real distributable unit, not just a working directory with ambition.

After publishing, verify both public indexes from a clean shell:

```
gemstone-publish-verify --gemstone-version 0.2.10 --native-version 0.1.2
```

The command checks project JSON, version-specific JSON, the simple index, and temporary-virtualenv installs for PyPI and TestPyPI.

Native Fast Path

The default install uses the pure-ctypes GCI backend:

```
python -m pip install gemstone-py
```

The optional native fast path installs the PyO3 extension package:

```
python -m pip install "gemstone-py[fast]"
```

Backend selection happens at import time. For comparisons, set `GEMSTONE_PY_GCI_BACKEND=ctypes` or `GEMSTONE_PY_GCI_BACKEND=native` before starting Python.

The runnable repository example is `examples/native_backend/check_backend.py`.

Recommended Adoption Path

If you are bringing `gemstone-py` into a project, a sensible order is:

1. start with `GemStoneConfig` + `GemStoneSession`
2. move most application work into `session_scope(...)`
3. use `PersistentRoot` first for obvious top-level state
4. introduce `GSCollection` only when indexed queries are justified
5. add `TypedOop[T]`, typed queries, or managed handles when the code benefits

   from clearer object identity and lifetime
6. use Flask or FastAPI providers once request lifecycles matter
7. install `gemstone-py[fast]` only after the ctypes path is working
8. add benchmarks and live tests once the system is real

That order keeps the learning curve honest without making the first week feel like a database theology seminar.

Manual Summary

If you remember only five things:

1. Be explicit about transaction policy.
2. Use `PersistentRoot` first unless you need something more specialized.
3. Use async wrappers for async frameworks, not for shared-session threading.
4. Use typed and managed OOP wrappers when raw integers hide too much intent.
5. The examples teach the package; the maintained workflows prove it.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.